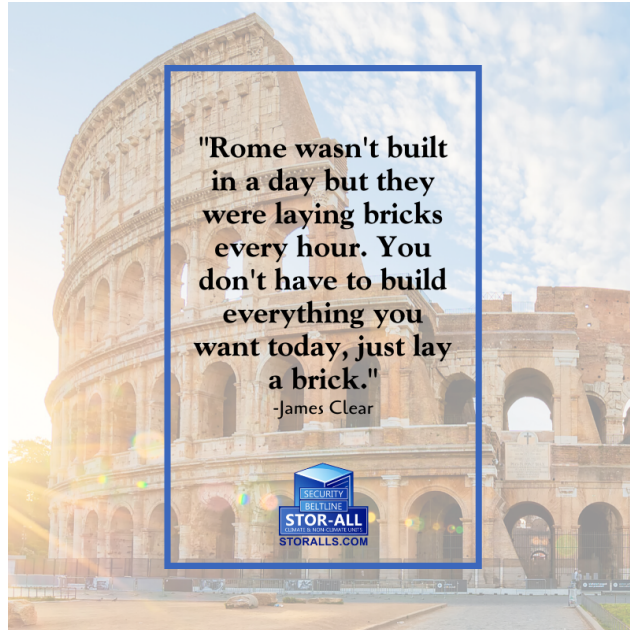


DSA for Fullstack / FrontEnd



Good
Evening

Agenda for Today

- Arrays
- Strings
- HashMap
- Stacks
- LinkedList
- Trees

Arrays

- prefix sum
- sliding window
- Kadane's Algo

* Prefix sum Technique (Range sum queries)

- Get the sum for continuous range of data
- Questions are usually around subarrays sum.

$$\begin{array}{ccccccc} A[] = \{ 2 & 3 & 6 & 1 & 5 & -2 & 3 \} \\ & 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ psum[] = 2 & 5 & 11 & 12 & 17 & 15 & 18 \end{array}$$

$psum[i]$ = sum of all elements from 0 to i .

$$psum[i] = psum[i-1] + A[i]$$

$$sum[2, 4] = psum[4] - psum[2-1]$$

$$sum[L, R] = \begin{cases} \text{if } (L > 0) & psum[R] - psum[L-1] \\ \text{else} & psum[R] \end{cases}$$

* Subarray sum = 0

$$A[] = \{ 2 \quad 3 \quad 6 \quad -3 \quad -3 \quad -2 \quad 3 \}$$

0 1 2 3 4 5 6

$$psum[] = 2 \quad 5 \quad 11 \quad 8 \quad 5 \quad 3 \quad 6$$



↑
if sum of

$$[0, 1] = 5$$

↑
if sum [0, 4] = 5



This concludes ele from 2 to 4 are
having their impact as 0.

Idea → For every psum value, go & check if
previously we have same value present at
some index. If so, we can return true.

* Subarray sum = k

$$A[] = \{ 1 \quad 2 \quad 3 \quad -2 \quad 4 \quad 6 \}$$

$$K = 5$$

$$psum(x) = 1 \quad 3 \quad 6 \quad 4 \quad 8$$

$$tar = -4 \quad -2 \quad 1 \quad -1 \quad 3$$

(x-k)

HS

1	3
4	6



Idea → For every $\text{sum} = x$, we will search if $x-k$ exist, If so, then we can claim, we have a subarr with $\text{sum} = k$.

* Kadane's Algo

$A[] = \{$	3	5	-10	-1	3	4	-5	12	$\}$
sum =	3	8	-2 ⁰	-1 ⁰	3	7	2	14	
ans =	3	8	8	8	8	8	8	14	
(-∞)									

```

sum = 0
ans = -∞

for (i=0; i<n; i++) {
    sum += A[i];
    if (sum > ans) ans = sum;
    if (sum < 0) sum = 0;
}

return ans;

```

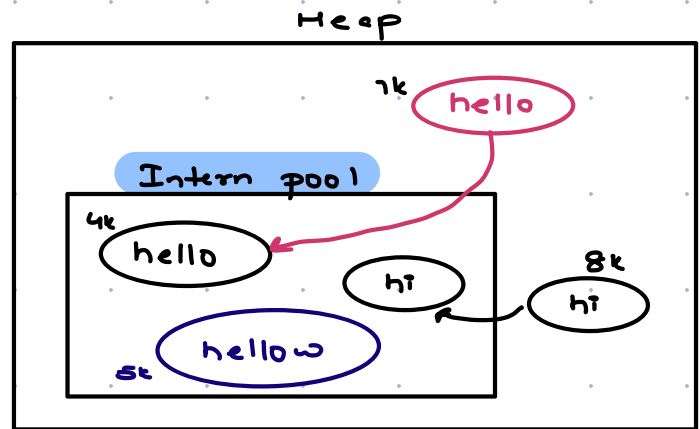
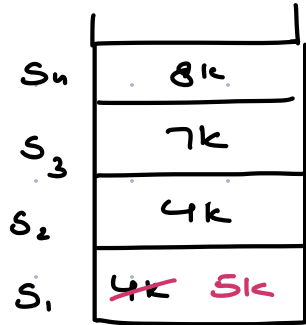
Tc: $O(N)$
Sc: $O(1)$

* **Strings** → Strings are immutable

String s₁ = "hello"

String s₂ = "hello"

s₁ = s₁ + "o";



String s₃ = new String("hello"); → Not generally followed

String s₄ = new String("hi")

* For making changes → StringBuilder

→ append()

→ insert(idx, ch)

→ reverse()

→ replace(idx, ch)

→ toString();

StringBuilder sb = new StringBuilder("hello");

- * Longest Common prefix
- * Reverse words in strings
- * Longest substring without repeat

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:

Input: strs = ["flower", "flow", "flight"]
Output: "fl"

```

len = strs[0].length();

for (i = 1; i < strs.length; i++)
    len = Math.min(len, strs[i].length());
    for (j = 0; j < len; j++) {
        if (strs[0].charAt(j) != strs[i].charAt(j))
            len = Min(len, j);
            break;
    }
}

return strs[0].substring(0, len)

```

TC: $O(N^2)$
SC: $O(1)$

* Reverse words in strings

String s = " _ _ I _ _ _ love _ coding _ _ _ "

str =

I	love	coding
0	1	2

```
class Solution {
    public String reverseWords(String s) {
        // Trim the input string to remove leading and trailing spaces
        String[] str = s.trim().split("\\s+");

        // Initialize the output string
        String out = "";
        StringBuilder sb = new StringBuilder();

        // Iterate through the words in reverse order
        for (int i = str.length - 1; i > 0; i--) {
            // Append the current word and a space to the output
            out += str[i] + " ";
        }

        // Append the first word to the output (without trailing space)
        return out + str[0];
    }
}
```

e d c b a
~~a~~ ~~b~~ c ~~d~~ e
0 1 2 3 4
↑↑
i

* Longest Substring without Repeat

→ Given a string str. Find longest substring without repeating characters.

str = " a b c d e b a k f "

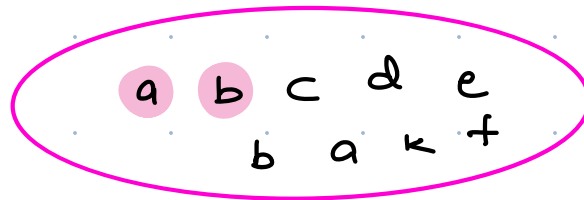
0 1 2 3 4 5 6 7 8

↑

5

TC: $O(N)$
SC: $O(1)$

HS $\Rightarrow$



len = 6

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int si = 0, c = 0;
        int ei = 0;
        int n = s.length();
        int len = 0;
        int[] farr = new int[256];
        while (ei < n) {
            if (farr[s.charAt(ei++)]++ > 0)
                c++;
            while (c > 0)
                if (farr[s.charAt(si++)]-- > 1)
                    c--;
            len = Math.max(len, ei - si);
        }
        return len;
    }
}
```

s
↓
abcabcbb"
↑
ei

No dup chara

↑
c = 0 ≠ 0

1	1	1	0	0
97	98	99	100	101
a	b	c	d	e

len = 3

if (farr[s.charAt(ei++)] ++ > 0)

↓

if (farr[s.charAt(ei)] > 0) c++;

farr[s.charAt(ei)] ++

ei++

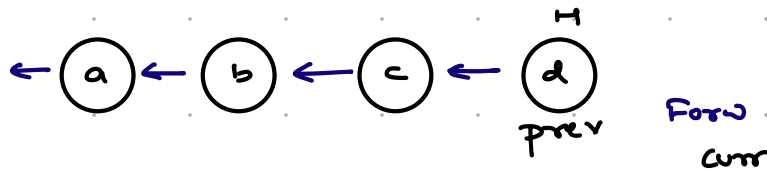
* **LinkedList** (majorly wants to see if you consider all edges or not)

→ Reverse a LL

→ Mid of LL

→ Cycle in LL

→ Handle pointers via dummy Node



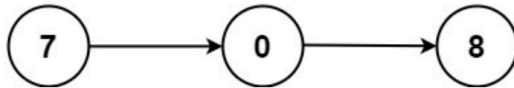
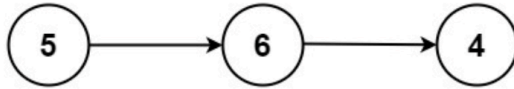
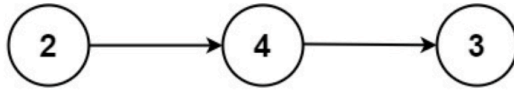
```
while (curr != Null)
{
    forw = curr.next;
    curr.next = prev;
    prev = curr;
    curr = forw;
}
return head = prev;
```

* Add two Numbers

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

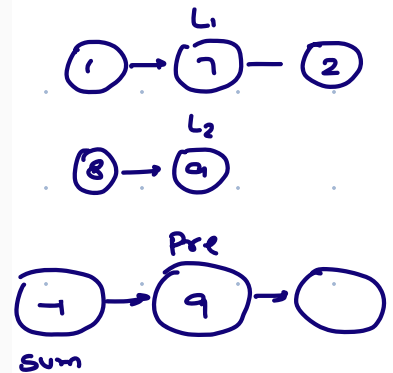
Explanation: 342 + 465 = 807.

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode sum = new ListNode(-1);
        ListNode presum = sum;

        int carry = 0;
        while(l1 != null || l2 != null){
            int s = carry;
            s += (l1 != null ? l1.val : 0) + (l2 != null ? l2.val : 0);

            carry = s / 10;
            s = s % 10;
            presum.next = new ListNode(s);
            presum = presum.next;

            if(l1 != null) l1 = l1.next;
            if(l2 != null) l2 = l2.next;
        }
        if(carry > 0) presum.next = new ListNode(carry);
        return sum.next;
    }
}
```



$$s = 0 + 1 + 8$$

$$= 9$$

$$9 / 10 = 0$$

$$s = 9 \% 10 = 9$$

Trees

